UNIVALI – Universidade do Vale do Itajaí Disciplina: Arquitetura e Organização de Computadores
Prática 01 – Programação de procedimentos em linguagem de montagem

<u>Lei atentamente as instruções abaixo</u>:

1. Esta atividade pode ser realizada individualmente ou em grupo de **até DOIS** alunos (no máximo).

2. A atividade consiste na implementação de **três programas** utilizando a linguagem de montagem do RISC-V32i.
3. *Os arquivos ASM de cada questão devem ser postados no AVA2 em um único arquivo .ZIP até o final da aula.*

4. Cada código-fonte deve ser escrito em um arquivo com extensão .asm e, **obrigatoriamente**, com um nome que identifique os membros do grupo e o nome do programa (ex. ***RingoStarr_JohnLennon_Programa_01.asm***). Obs: Todos os arquivos de todos os grupos serão reunidos em uma mesma pasta e, por isso, não use nomes como Programa_01.asm.

5. Cada código fonte deve conter um cabeçalho comentado que identifique a disciplina, a atividade, o programa e os nomes dos membros do grupo. Ex:

```
# Disciplina: Arquitetura e Organização de Processadores
# Atividade: Avaliação 01 – Programação em Linguagem de Montagem
# Programa 01
# Grupo: - Gale
#        - Lae'zel
```

6. Não serão aceitos trabalhos entregues em atraso.

7. O professor poderá solicitar que **qualquer aluno** do grupo faça uma **demonstração explicativa da execução dos códigos no RARS**.

8. **A implementação deverá apresentar resultados corretos para qualquer conjunto de dados.** Uma solução que **não execute corretamente** terá, automaticamente, um **desconto de 50% na nota**, sendo que o professor também avaliará a correção de segmentos específicos do código (controle de execução, acesso a memória,...).

9. Se forem identificados **trabalhos** com grau de **similaridade** que caracterize cópia (autorizada ou não) ou adaptação, a nota dos grupos será a **nota de um trabalho dividida** pelo número de grupos que entregou esses trabalhos similares.

CÓDIGOS DE LEITURA E ESCRITA BÁSICOS

```
.data
    string1: .asciz "Informe um valor: "
.text
    #comandos para escrever a string1 no console
    addi a7, zero, 4 #informa a operação de escrita te texto
    la   a0, string1 #coloca o endereço da string em a0
    ecall            #executa a operação

    #comandos para ler um inteiro do console
    addi a7, zero, 5 #informa a operação de leitura de inteiro
    ecall            #executa a operação
    add s0, zero, a0 #transfere o valor lido de a0 para s0

    #comandos para escrever um inteiro do console
    addi a7, zero, 1 #informa a operação de escrita de inteiro
    add a0, zero, s0 #coloca em a0 o inteiro que será escrito
    ecall            #executa a operação
```

CÓDIGOS DE DESVIO BÁSICOS

```
beq s0, s1, igual #se (s0 == s1) devia para igual
igual:
bne s0, s1, diferente #se (s0 != s1) devia para diferente
diferente:
bge  s0, s1, maior_igual #se (s0 >= s1) devia para maior_igual
maior_igual:
blt  s0, s1, menor #se (s0 < s1) devia para menor
menor:

#EXEMPLO: if(s0 == s1 and s0 == s2) {}
bne s0, s1, diferente
bne s0, s2, diferente
    #só entra aqui se forem iguais, senão vai ter desviado por serem diferentes
diferente:

#EXEMPLO: if(s0 == s1) {} else {}
bne s0, s1, diferente
    #se forem iguais não desvia
    jal zero, fim_if1 # esse comando faz saltar o else, pois não queremos executá-lo
diferente:
    #essa é a condição do else
fim_if1:
```

CÓDIGOS DE LAÇO DE REPETIÇÃO BÁSICO

```
#int t1=10;
#while(t1<t0){
#   t0++;
#}
addi t1,zero, 10
while:
  bge t0, t1, fim_while1    #t0<t1
  addi t0,t0,1              #t0++
  jal zero, while           #recomeça o laço
fim_while1:
nop                         #no operation
```

CÓDIGOS LEITURA E ESCRITA EM MEMÓRIA BÁSICOS

```
.data
    memoria1: .word 10 #vetor com apenas uma posição
.text

    #comando para ler da memória e colocar em s0
    la t0, memoria1 #coloca o início da memoria1 em t0
    lw s0, 0(t0)    #s0 = t0[0]

    #comando para colocar o valor 20 na memoria1
    addi s1, zero, 20
    la t0, memoria1 #coloca o início da memoria1 em t0
    sw s1, 0(t0)    #t0[0] = s1
```

PROGRAMA 01 (3,0 pontos)
Enunciado:
Faça uma função, não recursiva, que retorna o resto da divisão inteira sem utilizar o comando rem. Dica: utilizar subtração sucessiva.

Requisitos:

1. **Não pode utilizar pseudo instruções** com exceção do comando `la` que é uma pseudo instrução que não aprendemos a contornar ainda.

2. O procedimento deve utilizar as regras para preservar registradores aprendidas em aula.

3. Observe que esse é um procedimento folha, ou seja, não chama outros procedimentos.

PROGRAMA 02 (3,5 pontos)
Enunciado:
Faça uma função, não recursiva, que receba um valor e informe se o número é ou não primo.

```
int ehprimo (int a) {
    int i=2;
    if (a<2) {return 0;}
    while (i<a) {
        if (restoDivisao(a,i)==0) {return 0;}
        i++;
    }
    return 1;
}
```

Requisitos:

1. **Não pode utilizar pseudo instruções** com exceção do comando `la`.

2. A função deve utilizar a função criada no exercício 1.

3. A função deve utilizar as regras para preservar registradores aprendidas em aula.

4. Observe que é um procedimento não folha, ou seja, um procedimento que chama outro procedimento.

PROGRAMA 03 (3,5 pontos)
Enunciado:
Faça uma função **recursiva** que receba um número inteiro retorne a soma de todos os dígitos desse número.

```
int SomaDigitos(int n) {
    if (n < 10) {return n;}
        return (n % 10) + SomaDigitos(n / 10);
}
```

Requisitos:

1. **Não pode utilizar pseudo instruções** com exceção do comando `la` que é uma pseudo instrução que não aprendemos a contornar ainda.

2. A função deve utilizar a função criada no exercício 1.

3. A função deve utilizar as regras para preservar registradores aprendidas em aula.

4. Observe que é um procedimento recursivo, ou seja, um procedimento que chama ele mesmo.